

IN3050/IN4050 Mandatory Assignment 3: Unsupervised Learning

Name: Frida Berge

Username: fridatbe

Rules

Before you begin the exercise, review the rules at this website:

<https://www.uio.no/english/studies/examinations/compulsory-activities/mn-ifi-mandatory.html> , in particular the paragraph on cooperation. This is an individual assignment. You are not allowed to deliver together or copy/share source-code/answers with others. Read also the "Routines for handling suspicion of cheating and attempted cheating at the University of Oslo":

<https://www.uio.no/english/studies/examinations/cheating/index.html> We do not entirely prohibit the use of generative language models ("smart assistants" like ChatGPT, Llama, Claude or Copilot), but you must clearly acknowledge this at all times, following the UiO guidelines: https://www.uio.no/english/studies/resources/ai_student.html Note also that you must fully understand *all* the parts of you submissions, even if you got some help from a generative model. This will be tested during your peer review sessions (<https://www.uio.no/studier/emner/matnat/ifi/IN3050/v25/Peer%20review/>). By submitting this assignment, you confirm that you are familiar with the rules and the consequences of breaking them.

Delivery

Deadline: Tuesday, April 22, 2025, 23:59 CEST

Your submission should be delivered in Devilry. You may make several submissions in Devilry before the deadline, but only the last delivery will be read, so make sure to include all files in the last delivery. It is recommended to upload preliminary versions hours (or days) before the final deadline.

What to deliver?

We recommend you to solve all exercises in this Jupyter notebook (alternative 1), but it is also possible to work with a regular Python script (alternative 2).

Alternative 1: If you choose Jupyter, you should deliver the notebook. You should answer all questions and explain what you are doing in Markdown. Still, the code should be properly commented. The notebook should contain results of your runs. In addition, you also have to submit a PDF of your solution which shows the results of the runs.

Alternative 2: If you prefer not to use notebooks, you should deliver the code as Python scripts together with your run results and a PDF report where you answer all the questions and explain your work.

Here is a list of *absolutely necessary* (but not sufficient) conditions to get the assignment marked as passed:

- You must deliver your code (Python script or Jupyter notebook) you used to solve the assignment.
- The code used for making the output and plots must be included in the assignment.
- You must include example runs that clearly shows how to run all implemented functions and methods.
- All the code (in notebook cells or python main-blocks) must be runnable. If you have unfinished code that crashes, please comment it out and document what you think causes it to crash.
- You must also deliver a PDF of the code, outputs, comments and plots as explained above.

Make sure to include your name and username in the submitted files. Deliver one single compressed folder (.zip, .tgz or .tar.gz) which contains your complete solution.

Important: if you weren't able to finish the assignment, use the PDF report/Markdown to elaborate on what you've tried and what problems you encountered. Students who have made an effort and attempted all parts of the assignment will get a second chance even if they fail initially. This exercise will be graded PASS/FAIL.

Goals of the assignment

This assignment has two parts:

1. You will go through the basic theory of **Principal Component Analysis (PCA)** and implement PCA from scratch to compress and visualize data.
2. You will run **K-means clustering** using the `scikit-learn` toolkit and use PCA to visualize the results. You will also evaluate the output of K-means using a multi-class logistic regression classifier.

IN4050 students will have to do one extra part about tuning PCA to balance compression with information lost.

Tools

You may freely use code from the weekly exercises and the published solutions. In the first part about PCA you may **NOT** use ML libraries like `scikit-learn`. In the K-means part and beyond we encourage the use of `scikit-learn` to iterate quickly on the problems.

We will use the *numpy* library for performing matrix computations and the *pyplot* library for plotting data, as well as *scikit-learn* for K-means clustering. This assignment also

comes with a module called *data_assignment3* that you will use to import different (synthetic and real) datasets. Let's start by making sure that everything is installed.

In [249... `%pip install numpy matplotlib scikit-learn`

```
import data_assignment3
```

```
Requirement already satisfied: numpy in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (1.26.4)
Requirement already satisfied: matplotlib in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (3.10.0)
Requirement already satisfied: scikit-learn in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (1.6.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cyclery>=0.10 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: scipy>=1.6.0 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from scikit-learn) (1.15.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: six>=1.5 in c:\users\frida\anaconda3\envs\in3050_env\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

Part 1: Principal Component Analysis (PCA)

In this section, you will work with the PCA algorithm in order to understand its definition and explore its uses.

Part 1.1: Implementation

Here we implement the basic steps of PCA and we assemble them. We will only need functions from *numpy* for this part.

In [250... `import numpy as np`

Centering the Data

Implement a function with the following signature to center the data. Remember that every *feature* should be centered.

In [251...

```
def center_data(A):
    # INPUT:
    # A      [NxM] numpy data matrix (N samples, M features)
    #
    # OUTPUT:
    # X      [NxM] numpy centered data matrix (N samples, M features)

    #find the mean of each column
    A = A.astype(float)
    means = np.mean(A, axis=0)

    #subtracting the mean of each column from the column's elements
    for i in range(len(A[0])):
        A[:,i] -= means[i]
        # A[:,i] = np.round(A[:,i],2) #might remove

    return A

# center_data(data_assignment3.get_iris_data()[0])
```

Test your function checking the following assertion on *testcase* (absence of output means that the assertion holds):

In [252...

```
testcase = np.array([[3., 11., 4.3], [4., 5., 4.3], [5., 17., 4.5], [4, 13., 4.4]
answer = np.array([[-1., -0.5, -0.075], [0., -6.5, -0.075], [1., 5.5, 0.125], [0
np.testing.assert_array_almost_equal(center_data(testcase), answer)
```

Computing Covariance Matrix

Implement a function with the following signature to compute the covariance matrix. In order to get this at the correct scale, divide by $N - 1$, not N .

Note: Numpy provides a function `np.cov()` that does exactly this, but in this exercise we ask you to implement the code from scratch without using this function.

In [253...

```
def compute_covariance_matrix(A):
    # INPUT:
    # A      [NxM] numpy data matrix (N samples, M features)
    #
    # OUTPUT:
    # C      [MxM] numpy covariance matrix (M features, M features)
    A = center_data(A)
    n = len(A)
    X = A
    XT = A.T
    return (XT@X)/(n-1)

# data = data_assignment3.get_iris_data()[0]
# data = center_data(data)
# compute_covariance_matrix(data)
```

Test your function by comparing its output to the output of `np.cov()` :

```
In [254... test_array = np.array([[22.,11.,5.5],[10.,5.,2.5],[34.,17.,8.5],[28.,14.,7]])
answer = np.cov(np.transpose(test_array))
to_test = compute_covariance_matrix(test_array)
np.testing.assert_array_almost_equal(to_test, answer)
```

Computing eigenvalues and eigenvectors

Use the linear algebra package of `numpy` and its function `np.linalg.eig()` to compute eigenvalues and eigenvectors. Note that we only take the real part of the eigenvectors and eigenvalues. The covariance matrix *should* be a symmetric matrix, but the actual implementation in `compute_covariance_matrix()` can lead to small round off errors that lead to tiny imaginary additions to the eigenvalues and eigenvectors. These are purely numerical artifacts that we can safely remove.

Note: If you decide to NOT use `np.linalg.eig()` you must make sure that the eigenvalues you compute are of unit length!

```
In [255... def compute_eigenvalue_eigenvectors(A):
    # INPUT:
    # A    [DxD] numpy matrix
    #
    # OUTPUT:
    # eigval    [D] numpy vector of eigenvalues
    # eigvec    [DxD] numpy array of eigenvectors

    eigval, eigvec = np.linalg.eig(A)

    # Numerical roundoff can lead to (tiny) imaginary parts. We correct that here
    eigval = eigval.real
    eigvec = eigvec.real

    return eigval, eigvec
```

Test your function checking the following assertion on *testcase*:

```
In [256... testcase = np.array([[2, 0, 0], [0, 5, 0], [0, 0, 4]])
answer_eigval = np.array([2., 5., 4.])
answer_eigvec = np.array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])
x, y = compute_eigenvalue_eigenvectors(testcase)
np.testing.assert_array_almost_equal(x, answer_eigval)
np.testing.assert_array_almost_equal(y, answer_eigvec)
```

Sorting eigenvalues and eigenvectors

Implement a function with the following signature to sort eigenvalues and eigenvectors in descending order.

Remember that eigenvalue `eigval[i]` corresponds to eigenvector `eigvec[:, i]`.

```
In [257... def sort_eigenvalue_eigenvectors(eigval, eigvec):
    # INPUT:
```

```

# eigval    [D] numpy vector of eigenvalues
# eigvec    [DxD] numpy array of eigenvectors
#
# OUTPUT:
# sorted_eigval    [D] numpy vector of eigenvalues
# sorted_eigvec    [DxD] numpy array of eigenvectors

# print("input:")
# for i in range(len(eigval)):
#     print("i:",i)
#     print(eigval[i])
#     print(eigvec[:,i])

# indexes = {i: 0 for i in range(len(eigval))} #key - index in eigvec, value
indexes = [0]*len(eigval)
sorted_eigval = np.sort(eigval)[::-1] #descending order (argument by UIO GPT
sorted_eigvec = eigvec.copy() # making sorted_eigvec the same type and size

for i in range(len(sorted_eigval)):
    indexes[i] = list(eigval).index(sorted_eigval[i])

for i in range(len(sorted_eigval)):
    sorted_eigvec[:,i] = eigvec[:,indexes[i]]

return sorted_eigval, sorted_eigvec

```

Test your function checking the following assertion on *testcase*:

```

In [258... testcase = np.array([[2, 0, 0], [0, 5, 0], [0, 0, 4]])
answer_eigval = np.array([5., 4., 2.])
answer_eigvec = np.array([[0., 0., 1.], [1., 0., 0.], [0., 1., 0.]])
x, y = compute_eigenvalue_eigenvectors(testcase)
x, y = sort_eigenvalue_eigenvectors(x, y)
np.testing.assert_array_almost_equal(x, answer_eigval)
np.testing.assert_array_almost_equal(y, answer_eigvec)

```

PCA Algorithm

Implement a function with the following signature to compute PCA using the functions implemented above.

```

In [259... def pca(A, m):
    # INPUT:
    # A    [NxM] numpy data matrix (N samples, M features)
    # m    integer number denoting the number of learned features (m <= M)
    #
    # OUTPUT:
    # pca_eigvec    [Mxm] numpy matrix containing the eigenvectors (M dimensions
    # P             [Nxm] numpy PCA data matrix (N samples, m features)

    cov = compute_covariance_matrix(A)
    pca_data = center_data(A)
    dominantEvec = np.empty((len(cov),m))
    eval, evec = compute_eigenvalue_eigenvectors(cov)
    eval, evec = sort_eigenvalue_eigenvectors(eval, evec)

```

```

#choose n eigenvalues with the highest values beacuse they make a greater i
for i in range(m):
    dominantEvec[:,i] = evec[:,i] #on the first m indexes we find the highest

pca_eigvec = dominantEvec
P = pca_data@pca_eigvec

return pca_eigvec, P

```

Test your function checking the following assertion on *testcase*:

```

In [260... import pickle
testcase = np.array([[22., 11., 5.5], [10., 5., 2.5], [34., 17., 8.5]])
x, y = pca(testcase, 2)

answer1 = pickle.load(open('PCAanswer1.pkl', 'rb'))
answer2 = pickle.load(open('PCAanswer2.pkl', 'rb'))
test_arr_x = np.sum(np.abs(np.abs(x) - np.abs(answer1)), axis=0)
np.testing.assert_array_almost_equal(test_arr_x, np.zeros(2))
test_arr_y = np.sum(np.abs(np.abs(y) - np.abs(answer2)))
np.testing.assert_almost_equal(test_arr_y, 0)

```

Part 1.2: Understanding - How does PCA work?

We now use the PCA algorithm you implemented on a toy data set in order to understand its inner workings.

Loading the data

The module *data_assignment3* provides a small synthetic dataset of dimension [100x2] (100 samples, 2 features):

```

In [261... import data_assignment3

X = data_assignment3.get_synthetic_data()

```

Visualizing the data

Visualize the synthetic data using the function *scatter()* from the *matplotlib* library.

```

In [262... import matplotlib.pyplot as plt

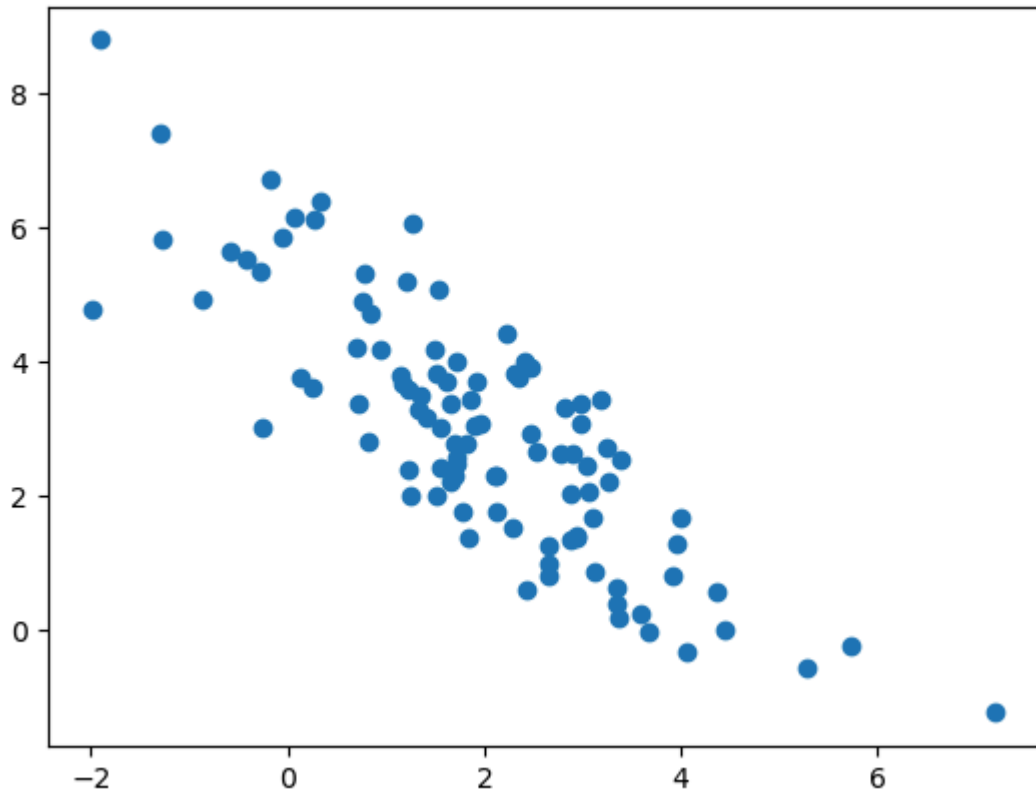
plt.scatter(X[:, 0], X[:, 1])

```

```

Out[262... <matplotlib.collections.PathCollection at 0x1b50d538830>

```

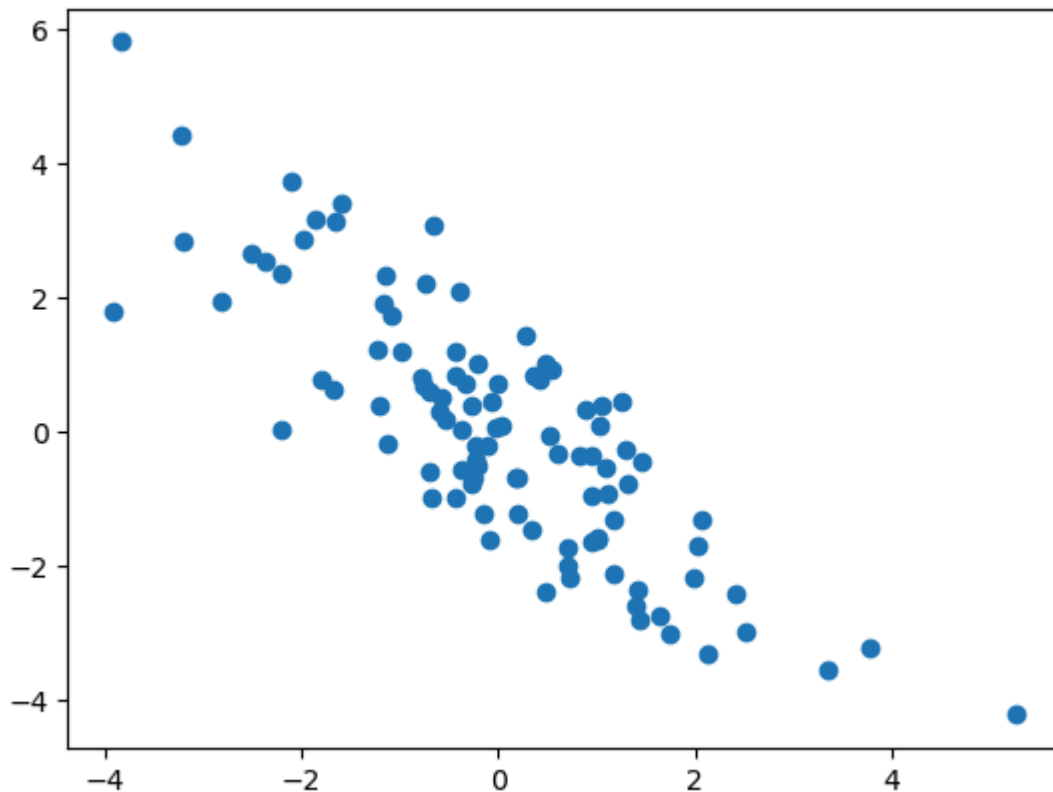


Visualize the centered data

Notice that the data visualized above is not centered on the origin (0,0). Use the function defined above to center the data, and then replot it.

```
In [263... centered_X = center_data(X)
plt.scatter(centered_X[:, 0], centered_X[:, 1])
```

```
Out[263... <matplotlib.collections.PathCollection at 0x1b50cbccbf0>
```



Visualize the first eigenvector

Visualize the vector defined by the first eigenvector. To do this you need:

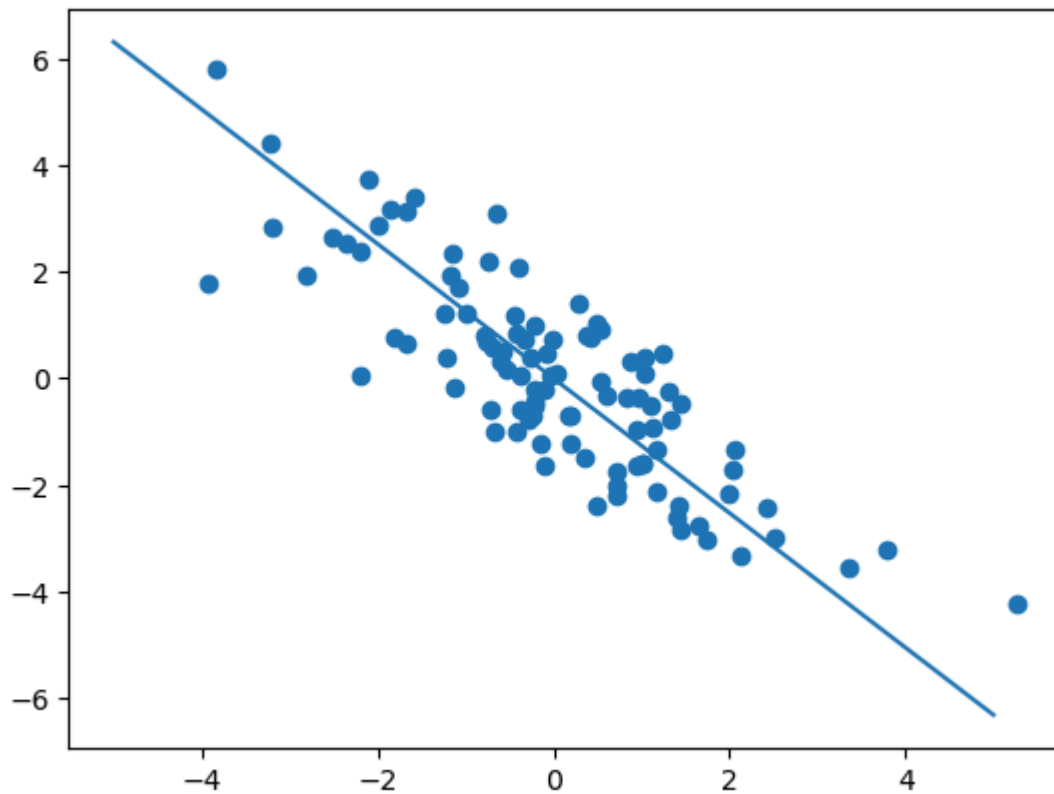
- Use the `PCA()` function to recover the eigenvectors
- Plot the centered data as done above
- The first eigenvector is a 2D vector (x_0, y_0) . This defines a vector with origin in $(0,0)$ and head in (x_0, y_0) . Use the function `plot()` from matplotlib to plot a line over the first eigenvector.

```
In [264...  pca_eigvec = pca(X,2)[0] #2 because the input has 2-dimensions, then I assume th
            first_eigvec = pca_eigvec[0]

            plt.scatter(centered_X[:, 0], centered_X[:, 1])

            x = np.linspace(-5, 5, 1000)
            y = first_eigvec[1]/first_eigvec[0] * x
            plt.plot(x, y)
```

```
Out[264...  [<matplotlib.lines.Line2D at 0x1b510e114f0>]
```

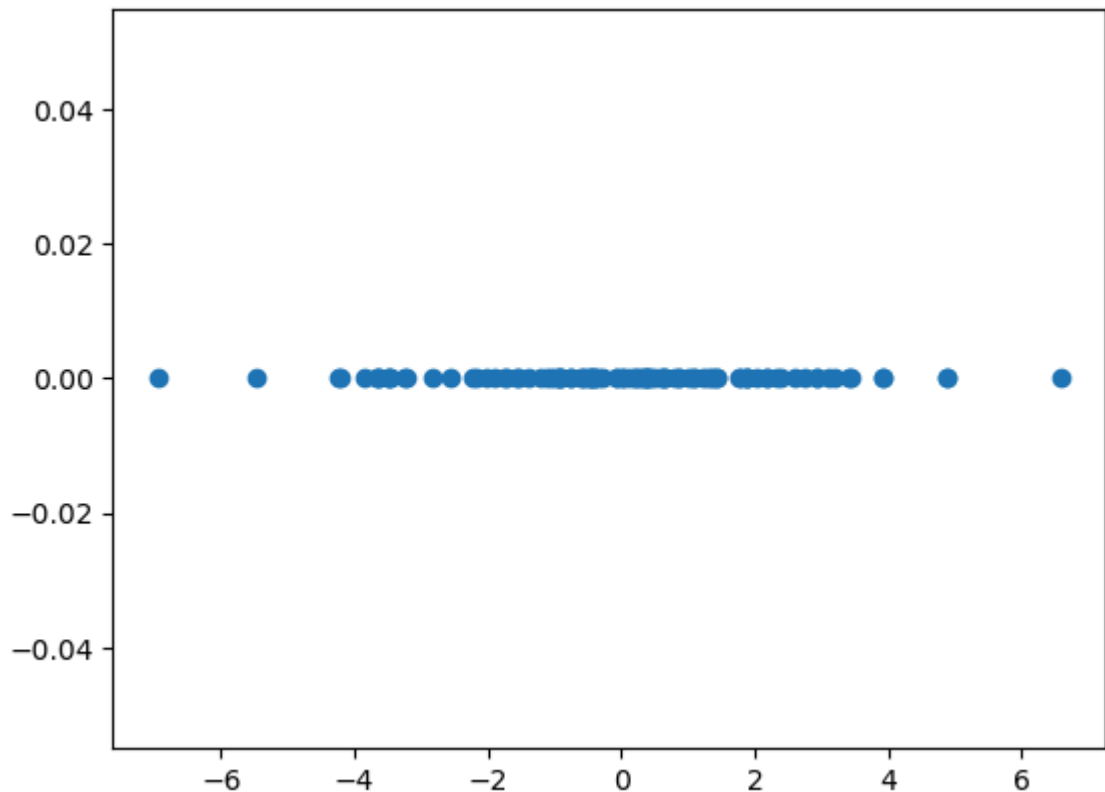


Visualize the PCA projection

Finally, use the `PCA()` algorithm to project on a single dimension and visualize the result using again the `scatter()` function.

```
In [265... ev, p = pca(X,1)
y_axis = [0]*len(p) #because eit is in one dimention y is always zero
plt.scatter(p, y_axis)
```

```
Out[265... <matplotlib.collections.PathCollection at 0x1b510e749e0>
```



Part 1.3: Evaluation - When are the results of PCA sensible?

So far we have used PCA on synthetic data. Let us now imagine we are using PCA as a pre-processing step before a classification task. This is a common setup with high-dimensional data. We explore when the use of PCA is sensible.

Loading the first set of labels

The function `get_synthetic_labeled_data_1()` from the module `data_assignment3` provides a first labeled dataset.

In [266... `X, y = data_assignment3.get_synthetic_labeled_data_1()`

Running PCA

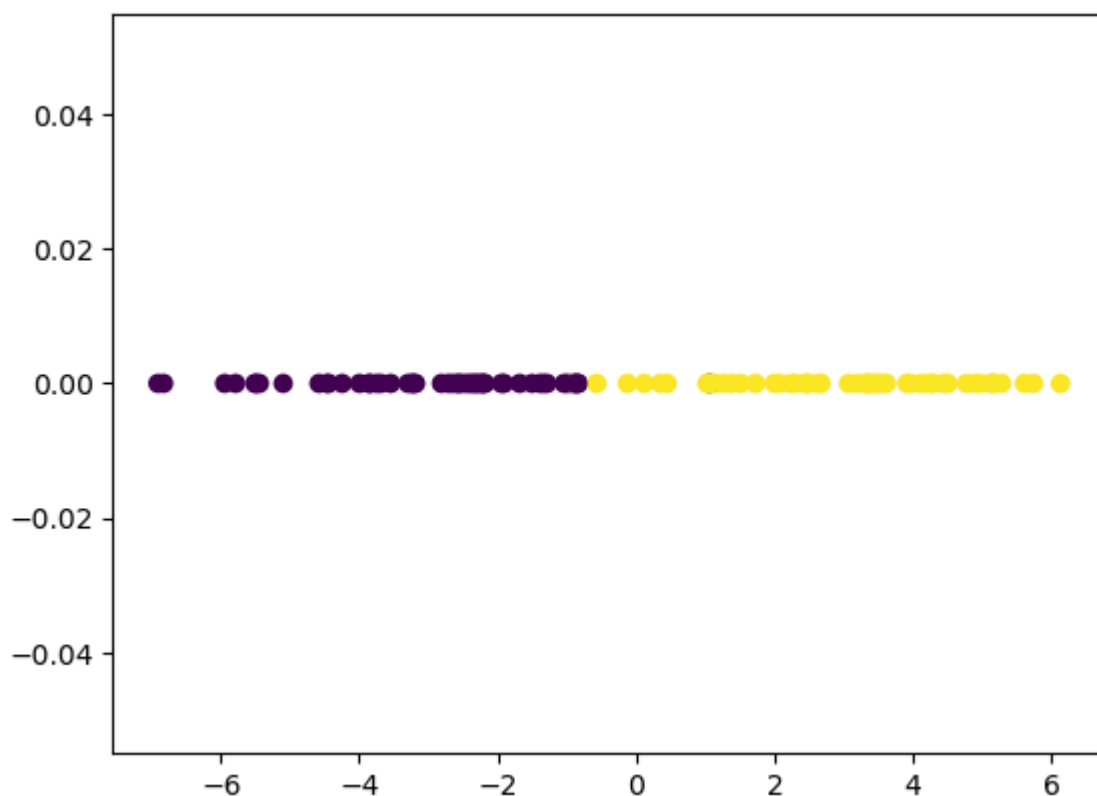
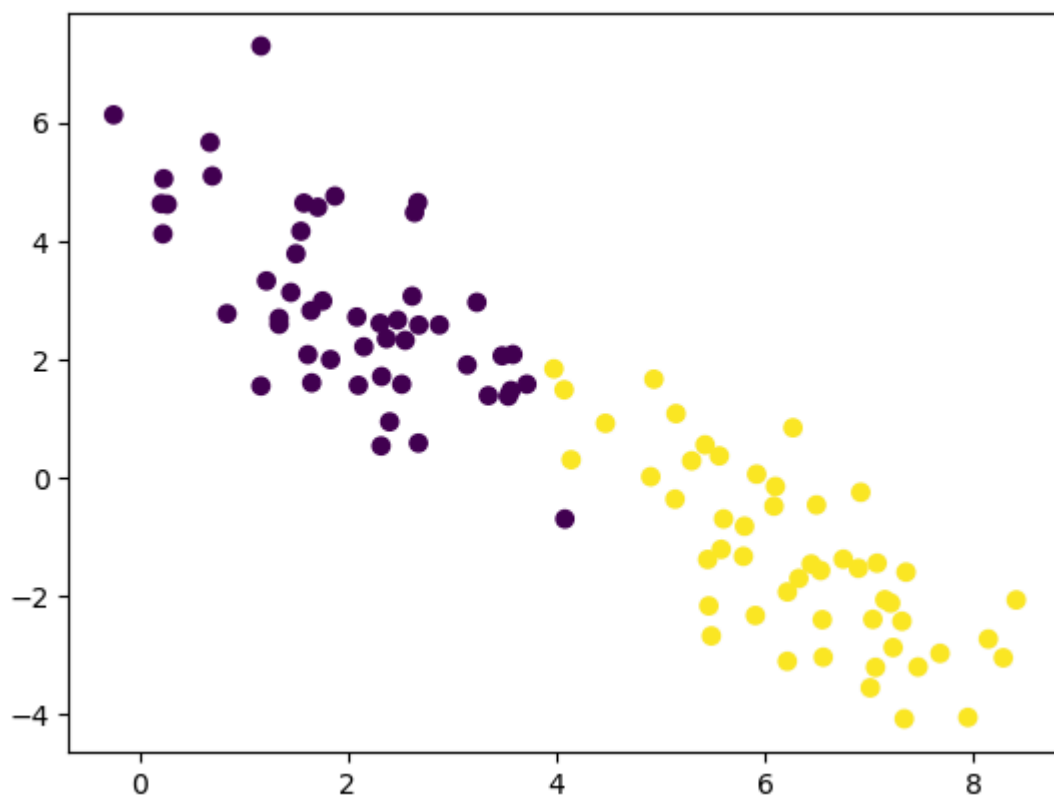
Process the data using the PCA algorithm and project it in one dimension. Plot the labeled data using `scatter()` before and after running PCA. Comment on the results.

In [267...

```
# before PCA
plt.scatter(X[:, 0], X[:, 1], c=y[:, 0]) #X has coordinates for the dots, y is t
plt.figure()

# after PCA
eigen, X_pca = pca(X, 1) #projecting into 1 dimension
Y_pca = [0]*len(X_pca)
plt.scatter(X_pca, Y_pca, c=y[:, 0]) #plotting Labeled data
plt.figure()
```

Out[267... <Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

Comment: The purple and yellow dots are separated like in the first figure. One purple dot should be in the yellow-dominated area, but i can't see it in the last figure, it might lay underneath the yellow dots. The density seem to be corresponding the density in the same parts in both figures. The data is now centered around origo

Loading the second set of labels

The function `get_synthetic_labeled_data_2()` from the module `data_assignment3` provides a second labeled dataset.

In [268... `X, y = data_assignment3.get_synthetic_labeled_data_2()`

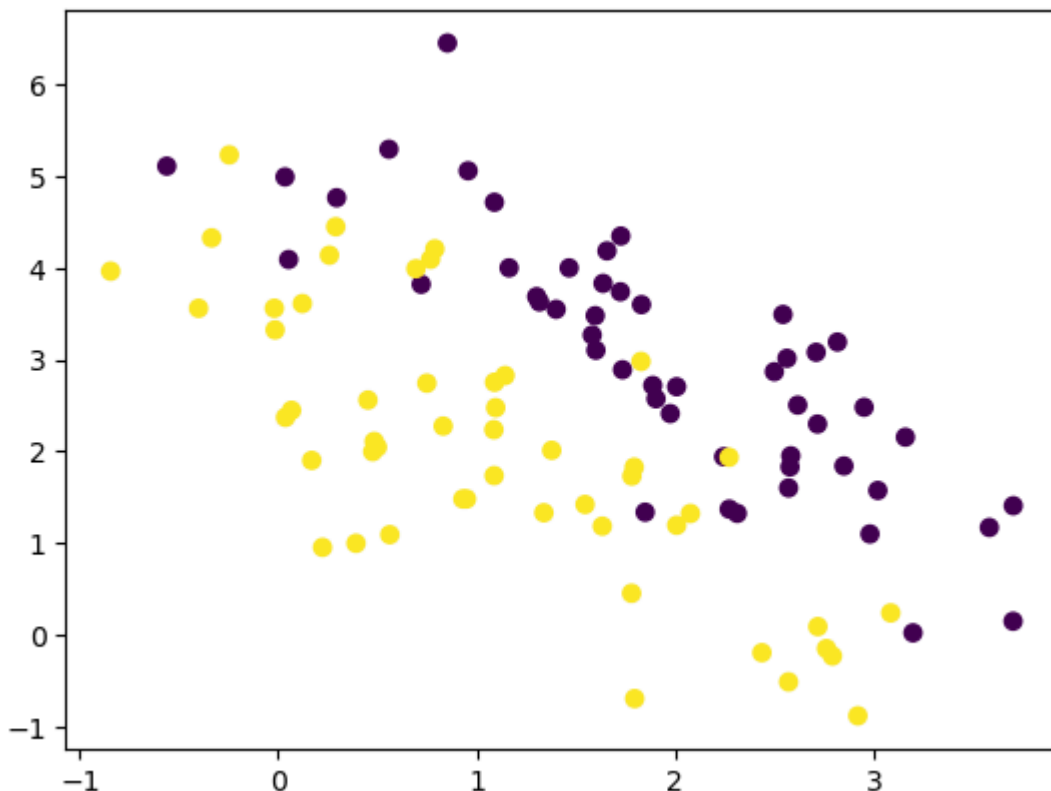
Running PCA

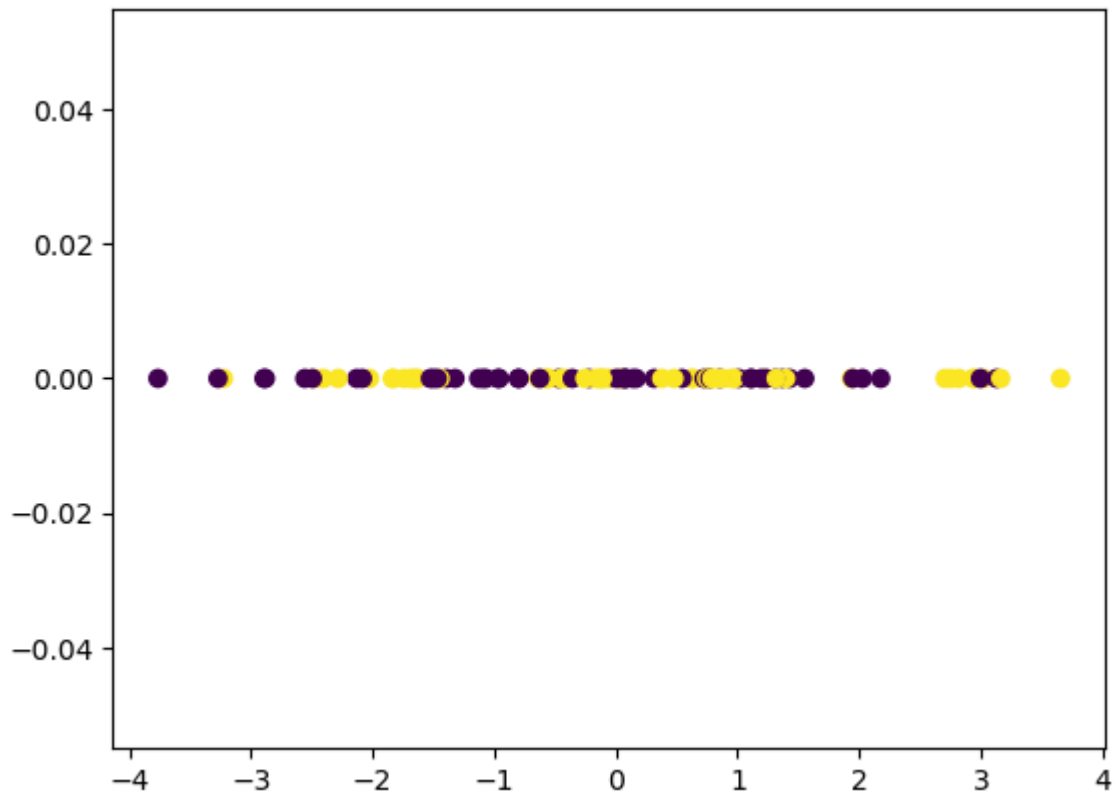
As before, process the data using the PCA algorithm and project it in one dimension. Plot the labeled data using `scatter()` before and after running PCA. Comment on the results.

In [269... `# before PCA`
`plt.scatter(X[:, 0], X[:, 1], c=y[:, 0]) #X has coordinates for the dots, y is t`
`plt.figure()`

`# after PCA`
`eigen, X_pca = pca(X, 1) #projecting into 1 dimation`
`Y_pca = [0]*len(X_pca)`
`plt.scatter(X_pca, Y_pca, c=y[:, 0]) #plotting labeled data`
`plt.figure()`

Out[269... <Figure size 640x480 with 0 Axes>





<Figure size 640x480 with 0 Axes>

Comment: We can see that the dots are more mixed in both figures compared to the set from previous task. It looks like the dot furthest to the right should be purple, but in the second figure it is yellow

How would the result change if you were to consider only the second eigenvector? What about if you were to consider both eigenvectors?

If we only choose the ***second eigenvector*** in stead of the first, the variance will be lower, but the classification will be more correct because the datapoints are classified in along the first eigenvector, but when the dataoints are projected on the second eigenvector normal to PCA 1, we can see a clear separation of the purple and yellow dots

If we consider ***both eigenvectors*** the PCA will project the dots into a 2-dimentional space centered around origo, and rotated around PCA 1 and PCA 2

Part 1.4: Case study 1 - PCA for visualization

The [iris flower dataset](#) is one of the oldest and best known data collections used for machine learning. It consists of 50 samples from each of three species of iris flowers (*Iris setosa*, *Iris virginica* and *Iris versicolor*). Four features were measured from each sample: sepal length, sepal width, petal length and petal width, all in centimeters.

Visualizing a 4-dimensional dataset is impossible; therefore we will use PCA to project our data in 2 dimensions and visualize it.

Loading the data

The function `get_iris_data()` from the module `data_assignment3` returns the *iris* dataset. It returns a data matrix of dimension [150x4] and a label vector of dimension [150].

```
In [270... X, y = data_assignment3.get_iris_data()
```

Visualizing the data by selecting features

Try to visualize the data (using label information) by randomly selecting two out of the four features of the data. You may try different pairs of features.

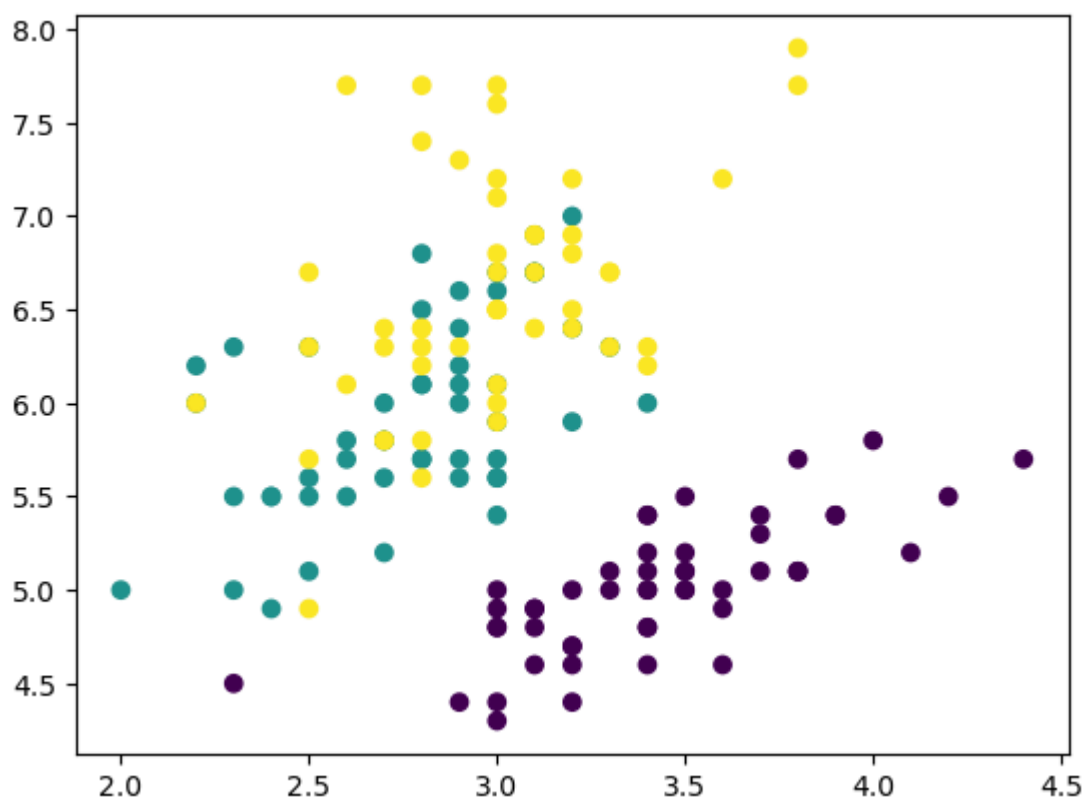
```
In [271... import random
np.random.seed()

features = [0,1,2,3]
f1 = features.pop(np.random.randint(0,3))
f2 = features.pop(np.random.randint(0,2))

print(f"Random features: {f1} and {f2}")
plt.scatter(X[:, f1], X[:, f2], c=y)
plt.figure()
```

Random features: 1 and 0

Out[271... <Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

Visualizing the data by PCA

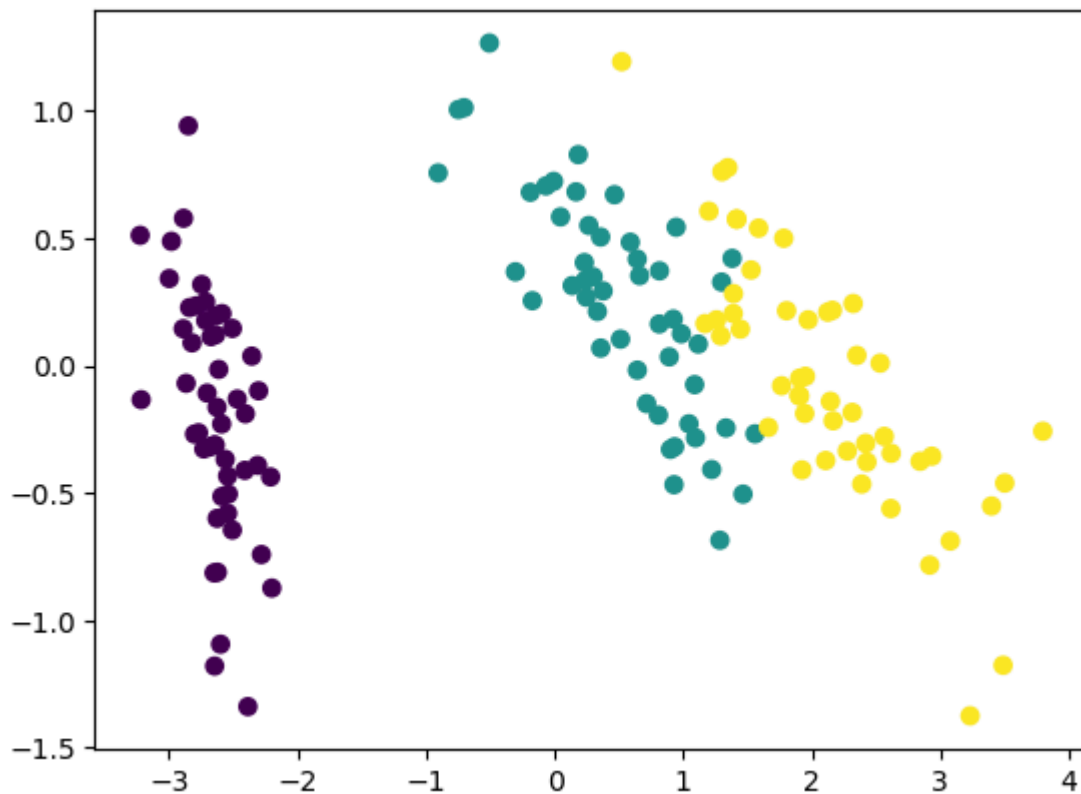
Process the data using PCA and visualize it (using label information). Compare with the previous visualization and comment on the results.

```
In [272... # chosen_features = X[:, [f1, f2]]
eigen_vecs, P = pca(X,2)

print(f"Most dominant features")
plt.scatter(P[:, 0], P[:, 1], c=y)
plt.figure()
```

Most dominant features

Out[272... <Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

Comment: The data is now centered around origo and might get rotated because the eigenvectors don't follow the x and y coordinate axes, so when it projects the datapoints onto the PCA 1 and PCA 2 it has to rotate/transform the data so that the intersection of PCA 1 and PCA 2 is at origo and the axes are parallel to the starting x-axis and z-axis

Part 1.5: Case study 2 - PCA for compression

We now consider the *faces in the wild (lfw)* dataset, a collection of pictures (N=1280) of people. Each pixel in the image is a feature (M=2914).

Loading the data

The function `get_lfw_data()` from the module `data_assignment3` returns the *lfw* dataset. It returns a data matrix of dimension [1280x2914] and a label vector of dimension [1280]. It also returns two parameters, *h* and *w*, reporting the height and the width of the images (these parameters are necessary to plot the data samples as images). Beware, it might take some time to download the data. Be patient :)

```
In [273... X, y, h, w = data_assignment3.get_lfw_data()
```

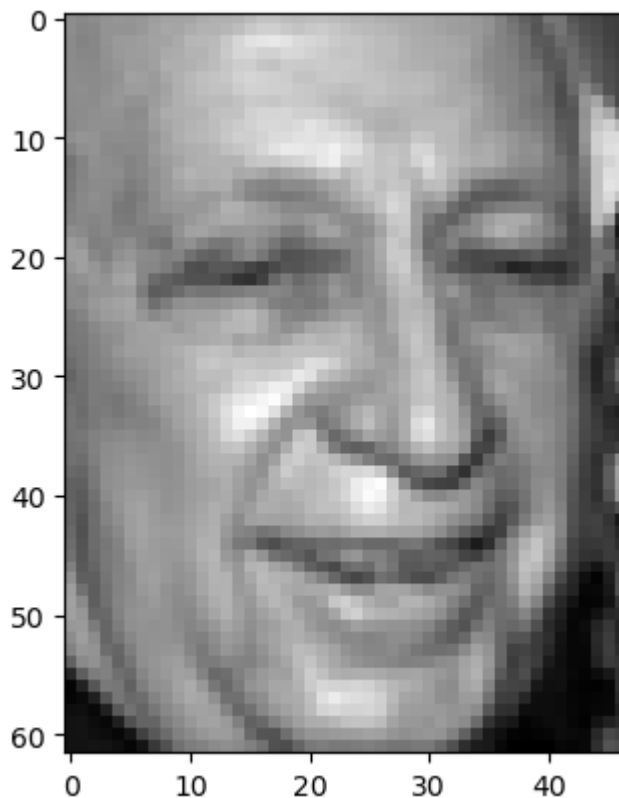
Inspecting the data

Choose one datapoint to visualize (first coordinate of the matrix X) and use the function `imshow()` to plot and inspect some of the pictures.

Notice that the first argument of `imshow` is the image to be plotted; the image must be provided as a rectangular matrix, therefore we reshape a sample from the matrix X to have height h and width w . The parameter `cmap` specifies the color coding; in our case we will visualize the image in black-and-white with different gradations of grey.

```
In [274... image_number = 4      # display the 4th image of the collection
plt.imshow(X[image_number, :].reshape((h, w)), cmap=plt.cm.gray)
```

```
Out[274... <matplotlib.image.AxesImage at 0x1b50cd458e0>
```



Implementing a compression-decompression function

Implement a function that first uses PCA to project samples in low-dimensions, and then reconstruct the original image.

Hint: Most of the code is the same as the previous `PCA()` function you implemented.

```
In [275... def encode_decode_pca(A, m):
    # INPUT:
    # A      [NxM] numpy data matrix (N samples, M features)
    # m      integer number denoting the number of learned features (m <= M)
    #
    # OUTPUT:
```

```

# Ahat [NxM] numpy PCA reconstructed data matrix (N samples, M features)

#from pca: encode (dimention reduction)
cov = compute_covariance_matrix(A)
means = np.mean(A, axis=0) #need the means Later
pca_data = center_data(A)
dominantEval = np.empty((len(cov),m))
eval, evec = compute_eigenvalue_eigenvectors(cov)
eval, evec = sort_eigenvalue_eigenvectors(eval, evec)
for i in range(m):
    dominantEval[:,i] = evec[:,i]
pca_eigvec = dominantEval
P = pca_data@pca_eigvec

#new for this task: decode
reshaped = P@pca_eigvec.T #reshape so it has the same shape as the input
ahat = reshaped+means #decenter the data so it reflects the input

return ahat

```

Compressing and decompressing the data

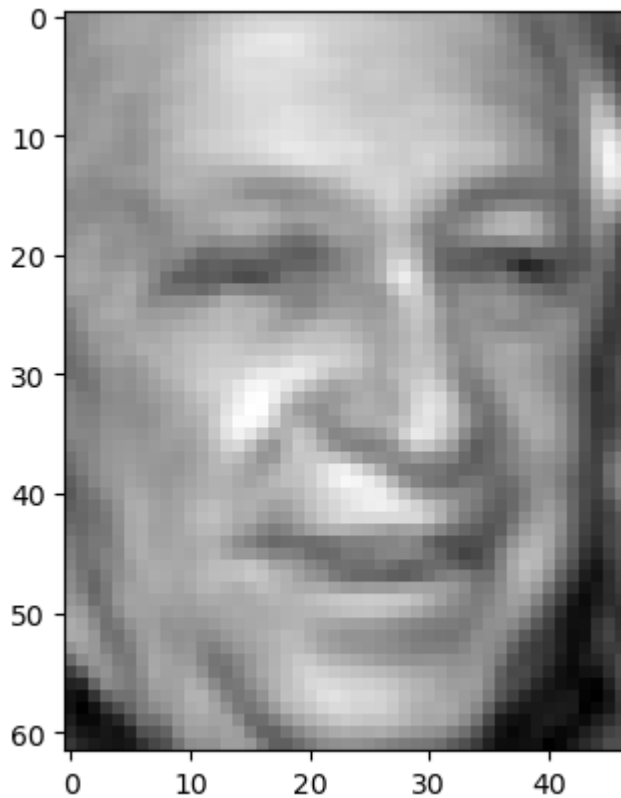
Use the implemented function to encode and decode the data by projecting on a lower dimensional space of dimension 200 (m=200).

In [276... Xhat = encode_decode_pca(X, 200)

Inspecting the reconstructed data

Use the function `imshow` to plot and compare original and reconstructed pictures. Comment on the results.

In [277... image_number = 4 # display the 4th image of the collection
plt.imshow(Xhat[image_number, :].reshape((h, w)), cmap=plt.cm.gray) #my reconstr
plt.show()
plt.imshow(X[image_number, :].reshape((h, w)), cmap=plt.cm.gray)
plt.show()



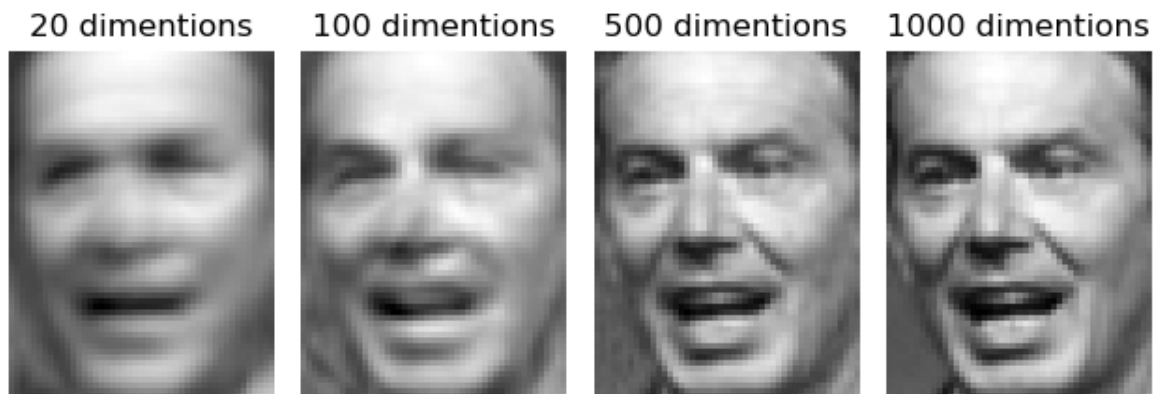
Comment: Pretty similar, I can see that some of the details are lost like the color of the right eye is not as dark as the original, and the same for the mouth

Evaluating different compressions

Use the previous setup to generate compressed images using different values of low dimensions in the PCA algorithm (e.g.: 20, 100, 200, 500, 1000). Plot and comment on the

results. You can use `plt.subplot(n_rows, n_cols, position)` and `plt.title(titlestring)` to get a nice plot.

```
In [278... n = [20,100,500,1000]
for i in range(4):
    plt.subplot(1, 4, i+1)
    Xhat = encode_decode_pca(X, n[i])
    plt.imshow(Xhat[1, :].reshape((h, w)), cmap=plt.cm.gray)
    plt.title(f"{n[i]} dimentions")
    plt.axis('off')
plt.tight_layout()
plt.show()
```



Comment: You can clearly see that more dimentions keeps more details than the reconctruction from lower dimentions

Part 1.6: PCA tuning (compulsory for Master students only)

If we use PCA for compression or decompression, it may be not trivial to decide how many dimensions to keep. In this section we review a principled way to decide how many dimensions to keep.

The number of dimensions to keep is the only *hyper-parameter* of PCA. A method designed to decide how many dimensions/eigenvectors is the *proportion of variance*:

$$\text{POV} = \frac{\sum_{i=1}^m \lambda_i}{\sum_{j=1}^M \lambda_j},$$

where λ are eigenvalues, M is the dimensionality of the original data, and m is the chosen lower dimensionality.

Using the *POV* formula we may select a number m of dimensions/eigenvalues so that the proportion of variance is, for instance, equal to 95%.

Implement a new PCA for encoding and decoding that receives in input not the number of dimensions for projection, but the amount of proportion of variance to be preserved.

```
In [279... def encode_decode_pca_with_pov(A, p):
    # INPUT:
```

```
# A    [NxM] numpy data matrix (N samples, M features)
# p    float number between 0 and 1 denoting the POV to be preserved
#
# OUTPUT:
# Ahat [NxM] numpy PCA reconstructed data matrix (N samples, M features)
# m    integer reporting the number of dimensions selected

Ahat = None
m = None
return Ahat, m
```

Import the `lfw` dataset again. Use the implemented function to encode and decode the data by projecting on a lower dimensional space such that `POV=0.95`. Use the function `imshow` to plot and compare original and reconstructed pictures. Comment on the results.

```
In [280... X, y, h, w = data_assignment3.get_lfw_data()

Xhat, m = None, None
print("Selected dimensions:", m)

# Plot the images here
```

Selected dimensions: None

Comment: Enter your comment here.

Part 2: K-Means Clustering

In this section you will use the *k-means clustering* algorithm to perform unsupervised clustering. Then you will perform a qualitative assessment of the results.

Part 2.1: Applying K-Means and Qualitative Assessment

Importing scikit-learn library

We start importing the module `sklearn.cluster.KMeans` from the standard machine learning library `scikit-learn`.

```
In [351... from sklearn.cluster import KMeans
```

Loading the data

We will use once again the *iris* data set. Start by loading the dataset again.

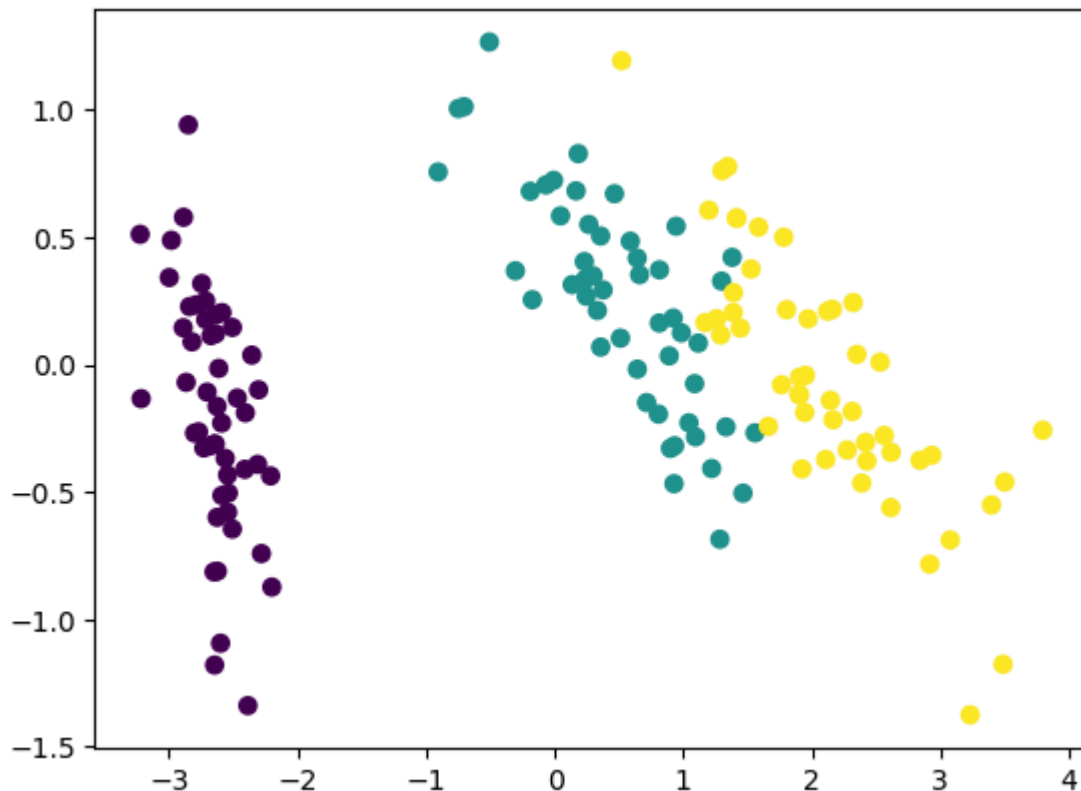
```
In [352... X, y = data_assignment3.get_iris_data()
```

Projecting the data using PCA

To allow for visualization, we project our data in two dimensions as we did previously. This step is not necessary, and we may want to try to use *k-means* later without the PCA pre-processing. But to start, we use PCA, as this will allow for an easy visualization.

```
In [353... X_pca, P = pca(X,2)
plt.scatter(P[:, 0], P[:, 1], c=y)
plt.figure()
```

Out[353... <Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

Running k-means

Use the class *KMeans* to fit and predict the output of the *k-means* algorithm on the projected data (note that we don't use the true labels `y` here). Run the algorithm using the following values of $k = \{2, 3, 4, 5\}$.

```
In [354... k_values = [2, 3, 4, 5]
clusters = [None]*len(k_values)
y_hats = [None]*len(k_values)
center = [0]*len(k_values)
labels = [0]*len(k_values)

for i in range(len(k_values)):
    # Initialize KMeans object with correct value for k
    cluster = KMeans(n_clusters = k_values[i])
    # Fit the algorithm to the training data and store the predictions in y_hats
    y_hats[i] = cluster.fit_predict(P)
    center[i] = cluster.cluster_centers_
    labels[i] = cluster.labels_ # same as y_hats
    clusters[i] = cluster
```

```
c:\Users\frida\anaconda3\envs\in3050_env\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
c:\Users\frida\anaconda3\envs\in3050_env\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
c:\Users\frida\anaconda3\envs\in3050_env\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
c:\Users\frida\anaconda3\envs\in3050_env\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

Qualitative assessment

Plot the results of running the k-means algorithm, compare with the true labels, and comment.

In [355...

```
subplot = 1

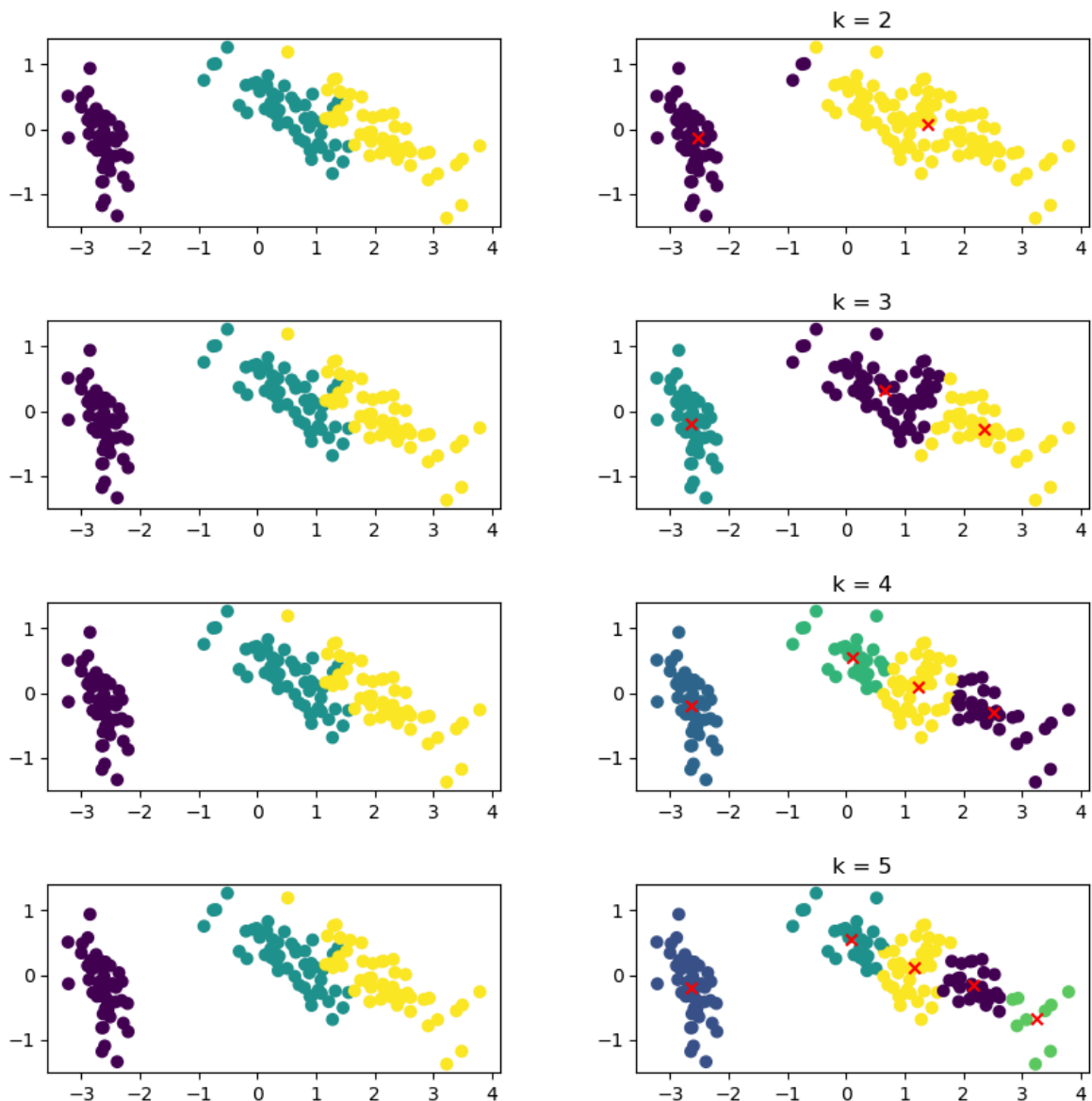
plt.figure(figsize=(10,10))

for i in range(len(y_hats)):

    plt.subplot(4, 2, subplot)
    plt.subplots_adjust(hspace=0.5, wspace=0.3)
    plt.scatter(P[:,0], P[:,1], c=y) #the true labels

    subplot += 1
    plt.subplot(4, 2, subplot)
    plt.title(f"k = {k_values[i]}")
    plt.scatter(P[:,0], P[:,1], c = y_hats[i])
    plt.scatter(center[i][:,0], center[i][:,1], c = 'red', marker = 'x') #the cl

    subplot+=1
plt.show()
```



Comment: From left column it is clear to see where the clusters should be, and we can see that the dots that are closest to each center have the same color. Going from 2 to 3 clusters we can see that some datapoints are not divided into the same clusters as the ones to the left, some of the yellow points on the plot to the left that are closer to the blue category belongs to the other category (corresponding to the yellow category on the right)

Part 2.2: Quantitative Assessment of K-Means

Above, we used k-means for clustering and assessed the results qualitatively by visualizing them. However, we often want to measure in a quantitative way how good a clustering is. To do this, we will use a classification task to **evaluate numerically how good the learned clusters are for all the different values of k you used above (2 to 5)**.

Informally, our evaluation will work as follows: For each of our clusterings ($k = 2$ to $k = 5$), we will try to learn a mapping from the identified clusters to the correct labels of our datapoints. The reason this can be a sensible evaluation, is that to learn a good

mapping, we have to have identified clusters that correspond to the actual classes in our data. We will in other words train 4 different classification models (one for each k value), where the input to our classifier is the cluster each datapoint belongs to, and the target is the correct class for this datapoint. In other words, **we aim to learn to classify datapoints as well as possible with the only information available to the classifier being the cluster that datapoint belongs to**. For some values of k , we will get poorly performing classifiers, indicating that this clustering has not revealed the correct class division in our data.

In practice, you will do the following: Reload the *iris* dataset. Import a standard `LogisticRegression` classifier from the module `sklearn.linear_model`. Use the k -means representations learned previously (`yhats[2], ..., yhats[5]`) and the true label to train the classifier. Evaluate your model on the training data (we do not have a test set, so this procedure will assess the model fit instead of generalization) using the `accuracy_score()` function from the `sklearn.metrics` module. Plot a graph showing how the accuracy score varies when changing the value of k . Comment on the results.

- Train a Logistic regression model using the first two dimensions of the PCA of the iris data set as input, and the true classes as targets.
- Report the model fit/accuracy on the training set.
- For each value of K :
 - One-Hot-Encode the classes outputed by the K -means algorithm.
 - Train a Logistic regression model on the K -means classes as input vs the real classes as targets.
 - Calculate model fit/accuracy vs. value of K .
- Plot your results in a graph and comment on the K -means fit.

```
In [372... from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import OneHotEncoder

#reload the iris dataset
X, y = data_assignment3.get_iris_data()
eigenvec, P = pca(X,2)

#Logistic regression classifier
inp = P #two dimentions of the PCA
target = y #true classes
logreg = LogisticRegression()
logreg.fit(inp, target)
pred = logreg.predict(inp)
acc = accuracy_score(target, pred)
print(f"Accuracy on the training set: {np.round(acc*100,2)}%")

#For K-mean of different sizes
x_axis = []
y_axis = []
onehot_encoder = OneHotEncoder() #from ChatGPT
for k in range(len(k_values)):
```

```

onehot = onehot_encoder.fit_transform(y_hats[k].reshape(-1,1)).toarray() #fr

logreg = LogisticRegression()
logreg.fit(onehot, target)
pred = logreg.predict(onehot)
kacc = accuracy_score(pred, target)

print(f"Accuracy when K = {k_values[k]}: {np.round(kacc*100,2)}%")
x_axis.append(k_values[k])
y_axis.append(kacc)

#plot in graph
plt.plot(x_axis,y_axis)
plt.xlabel("K")
plt.ylabel("Accuracy")
plt.grid(True)
plt.show()

```

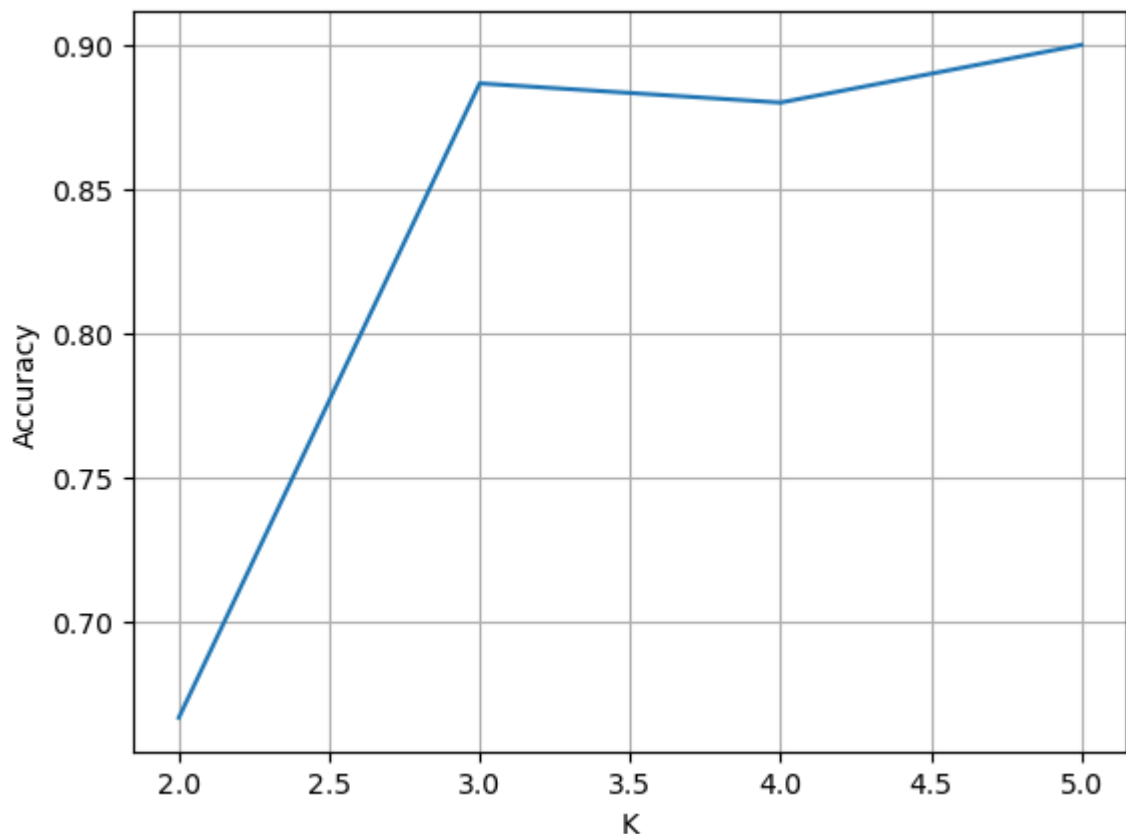
Accuracy on the training set: 96.67%

Accuracy when K = 2: 66.67%

Accuracy when K = 3: 88.67%

Accuracy when K = 4: 88.0%

Accuracy when K = 5: 90.0%



Comment: We can see that the accuracy increases with the number of clusters (K), but from 3 to 4 clusters it drops a little. It might gain better predictions with more clusters because with more clusters the datapoints are categorised into smaller groups. And the accuracy will be higher when more datapoints within a class are similar

Conclusions

In this notebook we studied **unsupervised learning** considering two important and representative algorithms: **PCA** and **k-means**.

First, we implemented the PCA algorithm step by step; we then ran the algorithm on synthetic data in order to see its working and evaluated when it makes sense to use it and when it doesn't. We then considered two typical uses of PCA: for **visualization** on the *iris* dataset, and for **compression-decompression** on the *lfw* dataset. We also looked at an additional question that arises when using PCA: the problem of **selection of hyper-parameters**, that is, how to select the optimal hyper-parameter of our algorithm for a particular task.

We then moved to consider the k-means algorithm. In this case we used the implementation provided by *scikit-learn* and applied it to another prototypical unsupervised learning problem: **clustering**. We processed the *iris* dataset with *k-means* and evaluated the results visually. We also considered the problem of **quantitative evaluation** of the results, that is, how to measure the performance or usefulness of k-means clustering on a downstream task (classifying the *iris* samples into their species).